
C: Como Programar

6ª Edição · Paul Deitel & Harvey Deitel

Capítulo 9

Entrada/Saída Formatada em C

Exercícios de Autorrevisão (9.1–9.3)

Resolvidos passo a passo, abordagem incremental
Capítulos 1 a 9

Construindo sobre os Capítulos Anteriores

No Capítulo 9, aprofundamos o que já conhecemos sobre `printf` e `scanf` desde o Cap. 2. Construindo sobre os fundamentos dos Caps. 1–8, incorporamos:

- Fluxos (*streams*) – toda E/S em C passa por `stdin`, `stdout`, `stderr`
- Especificadores de conversão completos – `%d`, `%i`, `%o`, `%u`, `%x`, `%X`, `%e`, `%E`, `%f`, `%g`, `%G`, `%c`, `%s`, `%p`, `%n`
- Flags de formatação – `-` (esquerda), `+` (sinal), espaço, `#`, `0`
- Modificadores de tipo – `h`, `l`, `L`
- Largura de campo e precisão – também via `*` para receber valores dinâmicos
- Sequências de escape – `\n`, `\t`, `\"`, `\\`, etc.
- Conjunto de varredura (*scan set*) no `scanf`: `%[...]`
- Caractere de supressão `*` no `scanf`

Boa prática de programação

Domínio fino de `printf` e `scanf` faz toda a diferença em programas de qualidade. Saída bem formatada economiza tempo do leitor; leitura precisa evita classes inteiras de bugs.

Contents

1	Exercício 9.1 – Preenchimento de Lacunas (22 itens)	2
2	Exercício 9.2 – Identificar e Corrigir Erros	3
3	Exercício 9.3 – Instruções	5

1. Exercício 9.1 – Preenchimento de Lacunas (22 itens)

Resposta detalhada

- a) Toda E/S é trabalhada na forma de **fluxos** (*streams*).
- b) O fluxo **de entrada-padrão** (*stdin*) está conectado ao teclado.
- c) O fluxo **de saída-padrão** (*stdout*) está conectado à tela.
- d) A formatação precisa de saída é obtida com a função **printf**.
- e) A string de controle de formato pode conter **especificadores de conversão, flags, larguras de campo, precisões e caracteres literais**.
- f) Tanto **d** quanto **i** podem ser usados para inteiros decimais com sinal.
- g) **o, u, x** (ou **X**) para inteiros sem sinal em formato octal, decimal e hexadecimal.
- h) Os modificadores **h** e **l** indicam **short** ou **long**.
- i) O especificador **e** (ou **E**) para notação exponencial.
- j) O modificador **L** para **long double**.
- k) Sem precisão especificada, **e, E, f** mostram **6** dígitos à direita do ponto decimal.
- l) **s** e **c** são usados para strings e caracteres.
- m) Todas as strings terminam no caractere **NULL** (`'\0'`).
- n) Largura e precisão dinâmicas via **%.*** – substituir por **asterisco (*)**.
- o) Flag **-** alinha à esquerda.
- p) Flag **+** exibe sinal explicitamente.
- q) Formatação precisa de entrada: função **scanf**.
- r) **Conjunto de varredura** (*scan set, %[...]*) varre por caracteres específicos.
- s) Especificador **i** no **scanf** para inteiros opcionalmente com sinal em qualquer base (decimal, octal ou hex).
- t) Para **double**: **le, lE, lf, lg** ou **lG**.
- u) **Caractere de supressão de atribuição (*)** lê e descarta.
- v) **Largura de campo** no **scanf** limita quantos caracteres ler.

2. Exercício 9.2 – Identificar e Corrigir Erros

Item (a)

Resposta detalhada

Código: `printf( "%s\n", 'c' );`

Erro: `%s` espera um ponteiro para `char` (string), não um único caractere.

Correção: usar `%c` para caractere, ou trocar `'c'` por `"c"`:

```
1 printf( "%c\n", 'c' );      /* caractere */
2 printf( "%s\n", "c" );     /* string */
```

Item (b)

Resposta detalhada

Código: `printf( "%.3f%", 9.375 );`

Erro: Tentativa de imprimir o caractere literal `%` sem usar a sequência `%%`.

Correção:

```
1 printf( "%.3f%%", 9.375 ); /* imprime 9.375% */
```

Item (c)

Resposta detalhada

Código: `printf( "%c\n", "Domingo" );`

Erro: `%c` espera um `char`, não um ponteiro para `char` (string).

Correção: usar `%1s` (string de largura 1):

```
1 printf( "%.1s\n", "Domingo" ); /* imprime D */
```

Ou desreferenciar o primeiro caractere:

```
1 printf( "%c\n", "Domingo"[ 0 ] );
```

Item (d)

Resposta detalhada

Código: `printf( ""Uma string entre aspas" );`

Erro: Aspas literais dentro de string sem escape.

Correção: usar `\`:

```
1 printf( "\"Uma string entre aspas\"" );
```

Item (e)

Resposta detalhada

Código: `printf( "%d%d", 12, 20 );`

Erro: A string de controle não está entre aspas.

Correção:

```
1 printf( "%d%d", 12, 20 );
```

Item (f)

Resposta detalhada

Código: `printf( "%c", "x" );`

Erro: Caractere `x` com aspas duplas (string) sendo passado para `%c` (que espera `char`).

Correção: usar aspas simples:

```
1 printf( "%c", 'x' );
```

Item (g)

Resposta detalhada

Código: `printf( "%s\n", 'Ricardo' );`

Erro: String entre aspas simples (`'Ricardo'`). Aspas simples são para caractere único.

Correção:

```
1 printf( "%s\n", "Ricardo" );
```

Boa prática de programação

Em C: **aspas simples** (`'x'`) para um caractere; **aspas duplas** (`"x"`) para uma string (mesmo com um único caractere – a string inclui o `'\0'`).

3. Exercício 9.3 – Instruções

Resposta detalhada

a) 1234 alinhado à direita em campo de 10:

```
1 printf( "%10d\n", 1234 );
```

b) 123.456789 em notação exponencial com sinal e 3 dígitos:

```
1 printf( "%+.3e\n", 123.456789 );
```

c) Ler double para number:

```
1 scanf( "%lf", &number );
```

Nota: lf no scanf para double.

d) Imprimir 100 em octal precedido de 0:

```
1 printf( "%#o\n", 100 ); /* imprime 0144 */
```

A flag # adiciona o prefixo 0 (octal) ou 0x/0X (hex).

e) Ler string para array string:

```
1 scanf( "%s", string );
```

f) Ler dígitos até encontrar não-dígito:

```
1 scanf( "%[0123456789]", n );
```

O conjunto de varredura lê apenas caracteres do conjunto especificado.

g) Usar x e y como largura e precisão dinâmicas:

```
1 printf( "%*.*f\n", x, y, 87.4573 );
```

h) Ler 3.5% e armazenar 3.5 em percent sem supressão:

```
1 scanf( "%f%%", &percent );
```

O %% na string de scanf significa: “espere um caractere literal % aqui”.

i) Imprimir long double 3.333333 com sinal, largura 20, precisão 3:

```
1 printf( "%+20.3Lf\n", 3.333333 );
```